

4.2 贪心法的正确性证明

最小延迟调度

问题:

客户集合 A , $\forall i \in A$, t_i 为服务时间, d_i 为要求完成时间, t_i, d_i 为正整数. 一个调度 $f: A \rightarrow \mathbb{N}$, $f(i)$ 为客户 i 的开始时间. 求最大延迟达到最小的调度, 即求 f 使得

$$\min_f \{ \max_{i \in A} \{ f(i) + t_i - d_i \} \}$$

$$\forall i, j \in A, i \neq j, f(i) + t_i \leq f(j) \\ \text{or } f(j) + t_j \leq f(i)$$

无等待时间

$$f(i) = f(i-1) + t_{i-1}$$

$$f(1) = 0$$



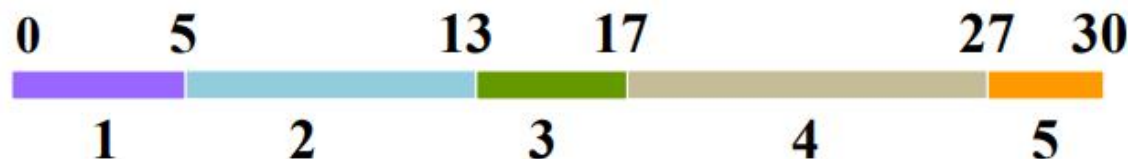
4.2 贪心法的正确性证明

实例：调度1

$A = \{1, 2, 3, 4, 5\}$, $T = \langle 5, 8, 4, 10, 3 \rangle$,
 $D = \langle 10, 12, 15, 11, 20 \rangle$

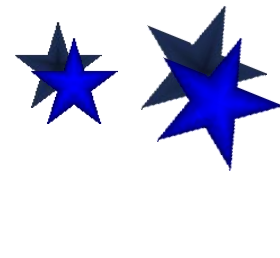
调度1：顺序安排

$f_1(1)=0, f_1(2)=5, f_1(3)=13, f_1(4)=17, f_1(5)=27$



各任务延迟：0, 1, 2, 16, 10;

最大延迟：16



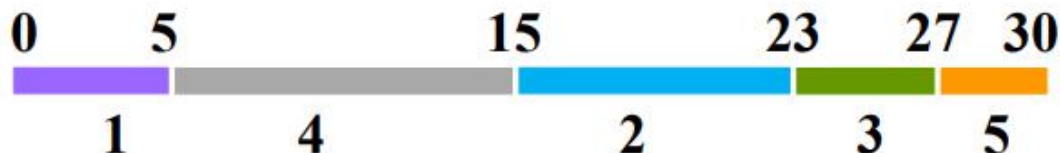
4.2 贪心法的正确性证明

更优的调度2

$A = \{1, 2, 3, 4, 5\}$, $T = \langle 5, 8, 4, 10, 3 \rangle$,
 $D = \langle 10, 12, 15, 11, 20 \rangle$

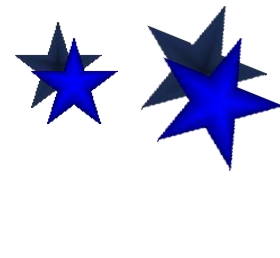
调度2：按截止时间从前到后安排

$f_2(1)=0, f_2(2)=15, f_2(3)=23, f_2(4)=5, f_2(5)=27$



各任务延迟：0, 11, 12, 4, 10;

最大延迟：12



4.2 贪心法的正确性证明

贪心策略

贪心策略1: 按照 t_i 从小到大安排

贪心策略2: 按照 $d_i - t_i$ 从小到大安排

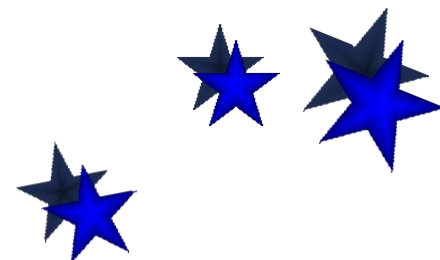
贪心策略3: 按照 d_i 从小到大安排

策略1 对某些实例得不到最优解.

反例: $t_1=1, d_1=100, t_2=10, d_2=10$

策略2 对某些实例得不到最优解.

反例: $t_1=1, d_1=2, t_2=10, d_2=10$



4.2 贪心法的正确性证明

策略3伪码

算法 Schedule

输入: A, T, D

输出: f

1. 排序 A 使得 $d_1 \leq d_2 \leq \dots \leq d_n$

2. $f(1) \leftarrow 0$

从0时
刻起

3. $i \leftarrow 2$

4. while $i \leq n$ do

5. $f(i) \leftarrow f(i-1) + t_{i-1}$

没有
空闲

6. $i \leftarrow i + 1$

设计思想: 按完成时间从早到晚安排任务, 没有空闲.

4.2 贪心法的正确性证明

交换论证：正确性证明

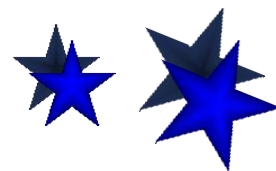
证明思路：

- 分析一般最优解与算法解的区别（成分,排列顺序不同）
- 设计一种转换操作（替换成分或交换次序），可以在有限步将任意一个普通最优解逐步转换成算法的解
- 上述每步转换都不降低解的最优性质

贪心算法的解的性质：

没有空闲时间，没有逆序。

逆序 (i, j) : $f(i) < f(j)$ 且 $d_i > d_j$



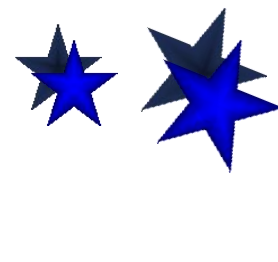
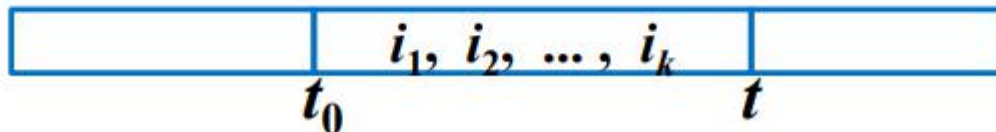
4.2 贪心法的正确性证明

引理

引理1 所有没有逆序、没有空闲时间的调度具有相同的最大延迟.

证: 设 f 没有逆序, 在 f 中具有相同完成时间 d 的客户 $i_1, i_2, \dots, i_k$ 连续安排, 其开始时刻为 t_0 , 完成这些任务的时刻是 t , 最大延迟为最后任务延迟 $t-d$, 与 $i_1, i_2, \dots, i_k$ 的排列次序无关.

$$t = t_0 + (t_{i_1} + t_{i_2} + \dots + t_{i_k})$$

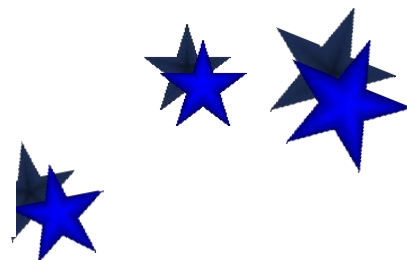


4.2 贪心法的正确性证明

证明要点

从一个没有空闲的最优解出发，逐步转变成没有逆序的解。根据引理 1，这个解和算法解具有相同的最大延迟。

- (1) 如果一个最优调度存在逆序，那么存在 $i < n$ 使得 $(i, i+1)$ 构成一个逆序，称为相邻的逆序。
- (2) 交换相邻逆序 i 和 j ，得到的解仍旧最优。
- (3) 每次交换后逆序数减 1，至多经过 $n(n-1)/2$ 次交换得到一个没有逆序的最优调度——等价于算法的解。



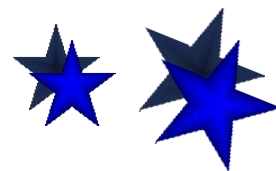
4.2 贪心法的正确性证明

交换相邻逆序仍旧最优

设 f_1 是一个任意最优解，存在相邻逆序 (i, j) 。交换 i 和 j 的顺序，得到解 f_2 。那么 f_2 的最大延迟不超过 f_1 的最大延迟。

理由：

- (1) 交换 i, j 与其他客户延迟时间无关
- (2) 交换后不增加 j 的延迟，但可能增加 i 的延迟
- (3) i 在 f_2 的延迟小于 j 在 f_1 的延迟，因此小于 f_1 的最大延迟 r



4.2 贪心法的正确性证明

i 在 f_2 的延迟不超过
 j 在 f_1 的延迟

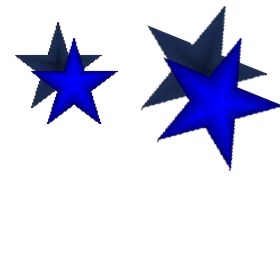


$$\text{delay}(f_2, i) = \underline{s + t_j + t_i} - d_i$$

$$\text{delay}(f_1, j) = \underline{s + t_i + t_j} - d_j$$

$$d_j < d_i$$

$$\text{delay}(f_2, i) < \text{delay}(f_1, j) \leq r$$

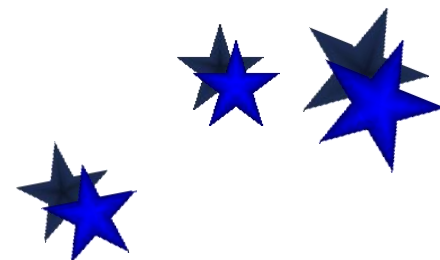


4.2 贪心法的正确性证明

小结

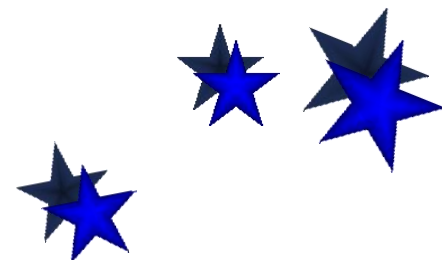
贪心法正确性证明方法：交换论证

- 分析算法解与一般最优解的区别, 找到把一般解改造成算法解的一系列操作(替换成份、交换次序)
- 证明操作步数有限
- 证明每步操作后的得到解仍旧保持最优



4.3 贪心法得不到最优解的处理

- 贪心法在解决组合优化问题时效率很高：取决于排序算法。
- 贪心法在解决某一问题时，某些实例可以得到最优解，但另外一些实例却不能得到正确的解。
- 若对所有的实例都能得到最优解，则算法的时间复杂度是指数级的，例如：背包问题。
- 贪心法可用于某些实际问题的处理——装载问题（0-1背包的子问题）



4.3 贪心法得不到最优解的处理

得不到最优解的处理

- 输入参数分析：
考虑输入参数在什么取值范围内使用贪心法可以得到最优解
- 误差分析：
估计贪心法——近似算法所得到的解与最优解的误差（对所有的输入实例在最坏情况下误差的上界）

第十章
介绍



4.3 贪心法得不到最优解的处理

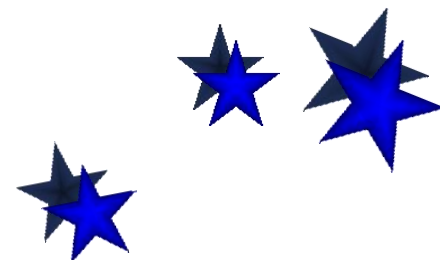
找零钱问题

问题 设有 n 种零钱，重量分别为 $w_1, w_2, \dots, w_n$ ，价值分别为 $v_1 = 1, v_2, \dots, v_n$ 。需要付的总钱数是 y 。不妨设币值和钱数都为正整数。问：如何付钱使得所付钱的总重最轻？

实例

$v_1=1, v_2=5, v_3=14, v_4=18, w_i=1, i=1,2,3,4.$
 $y=28$

最优解： $x_3=2, x_1=x_2=x_4=0$ ，总重2



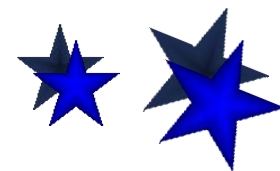
4.3 贪心法得不到最优解的处理

建模

令选用第 i 种硬币的数目是 x_i ,
 $i = 1, 2, \dots, n$

$$\min \left\{ \sum_{i=1}^n w_i x_i \right\}$$

$$\sum_{i=1}^n v_i x_i = y, \quad x_i \in \mathbf{N}, \quad i = 1, 2, \dots, n$$



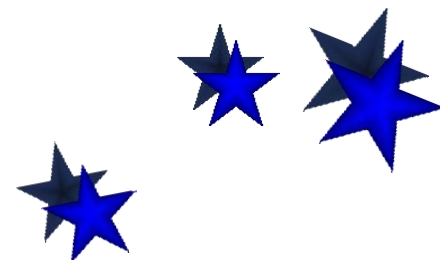
4.3 贪心法得不到最优解的处理

动态规划算法

设 $F_k(y)$ 表示用前 k 种零钱，总钱数为 y 的最小重量

$$F_k(y) = \min_{0 \leq x_k \leq \left\lfloor \frac{y}{v_k} \right\rfloor} \{F_{k-1}(y - v_k x_k) + w_k x_k\}$$

$$F_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y$$



4.3 贪心法得不到最优解的处理

贪心法

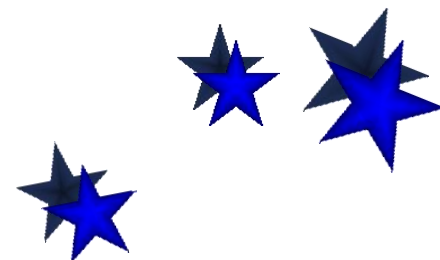
单位价值重量轻的货币优先，设

$$\frac{w_1}{v_1} \geq \frac{w_2}{v_2} \geq \dots \geq \frac{w_n}{v_n}$$

使用前 k 种零钱，总钱数为 y
贪心法的总重为 $G_k(y)$,

$$G_k(y) = w_k \left\lfloor \frac{y}{v_k} \right\rfloor + G_{k-1}(y \bmod v_k) \quad k > 1$$

$$G_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y$$



4.3 贪心法得不到最优解的处理

$n=1,2$ 贪心法是最优解

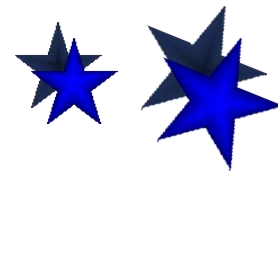
$n=1$ 只有一种零钱, $F_1(y) = G_1(y)$

$n=2$, x_2 越大, 得到的解越好

$$F_2(y) = \min_{0 \leq x_2 \leq \lfloor y/v_2 \rfloor} \{F_1(y - v_2 x_2) + w_2 x_2\}$$

$$\begin{aligned} & F_1(y - v_2(\underline{x_2 + \delta})) + w_2(\underline{x_2 + \delta}) \\ & - [F_1(y - v_2 \underline{x_2}) + w_2 \underline{x_2}] \\ = & [w_1(\underline{y - v_2 x_2 - v_2 \delta}) + \underline{w_2 x_2 + w_2 \delta}] \\ & - [w_1(\underline{y - v_2 x_2}) + \underline{w_2 x_2}] \\ = & -w_1 v_2 \delta + w_2 \delta \\ = & \delta(-w_1 v_2 + w_2) \leq 0 \end{aligned}$$

$$w_1 \geq w_2/v_2$$



4.3 贪心法得不到最优解的处理

判别条件

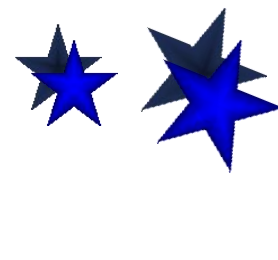
定理 对每个正整数 k , 假设对所有非负整数 y 有 $G_k(y) = F_k(y)$, 且存在 p 和 δ 满足

$$v_{k+1} = pv_k - \delta,$$

其中 $0 \leq \delta < v_k$, $v_k \leq v_{k+1}$, p 为正整数,

则下面的命题等价:

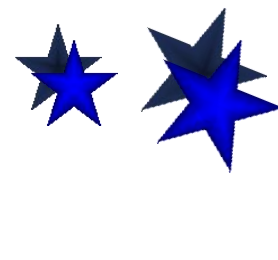
- (1) $G_{k+1}(y) = F_{k+1}(y)$ 对一切正整数 y ;
- (2) $G_{k+1}(pv_k) = F_{k+1}(pv_k)$;
- (3) $w_{k+1} + G_k(\delta) \leq pw_k$.



4.3 贪心法得不到最优解的处理

几点说明

- 根据条件(1)与(3)的等价性, 可以对 $k = 3, 4, \dots, n$, 依次利用条件(3)对贪心法是否得到最优解做出判别.
- 条件(3)验证 1 次需 $O(k)$ 时间, $k = O(n)$, 整个验证时间 $O(n^2)$
- 条件(2)是条件(1) 在 $y = pv_k$ 时的特殊情况. 若条件(1)成立, 显然有条件(2)成立. 反之, 若条件(2)不成立, 则条件(1)不成立, 钱数 $y = pv_k$ 恰好提供了一个贪心法不正确的反例.



4.3 贪心法得不到最优解的处理

验证实例

$$\begin{aligned} v_{k+1} &= pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+ \\ w_{k+1} + G_k(\delta) &\leq pw_k \end{aligned}$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1, i=1,2,3,4$. 对一切 y 有

$$G_1(y)=F_1(y), \quad G_2(y)=F_2(y).$$

验证 $G_3(y) = F_3(y)$

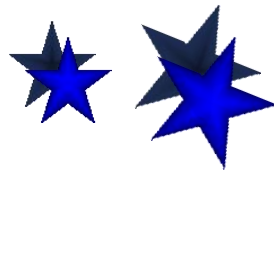
$$v_3 = pv_2 - \delta \Rightarrow p = 3, \delta = 1.$$

$$w_3 + G_2(\delta) = 1 + 1 = 2$$

$$pw_2 = 3 \times 1 = 3$$

$$w_3 + G_2(\delta) \leq pw_2$$

贪心法对于 $n=3$ 的实例得到最优解



4.3 贪心法得不到最优解的处理

验证实例

$$v_{k+1} = pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+ \\ w_{k+1} + G_k(\delta) \leq pw_k$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1,$

$i=1,2,3,4.$ 对一切 y 有

$$G_1(y)=F_1(y), G_2(y)=F_2(y), G_3(y)=F_3(y)$$

验证 $G_4(y) = F_4(y)$

$$v_4 = pv_3 - \delta \Rightarrow p=2, \delta=10$$

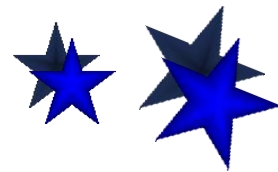
$$w_4 + G_3(\delta) = 1 + 2 = 3$$

$$pw_3 = 2 \times 1 = 2$$

$$w_4 + G_3(\delta) > pw_3$$

$$n=4, y=pv_3=28,$$

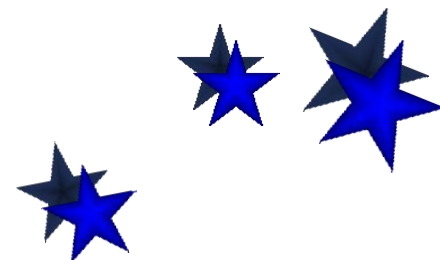
最优解: $x_3=2$, 贪心法: $x_4=1, x_2=2$



4.3 贪心法得不到最优解的处理

小结

- 贪心策略不一定得到最优解，在这种情况下可以有两种处理方法：
 - (1) 参数化分析：分析参数取什么值可得到最优解
 - (2) 估计贪心法得到的解在最坏情况下与最优解的误差
- 一个参数化分析的例子：找零钱问题



4.4 贪心法的典型应用

■ 最优前缀码——哈夫曼编码

二元前缀码

- 二元前缀码

用0-1字符串作为代码表示字符，要求任何字符的代码都不能作为其它字符代码的前缀

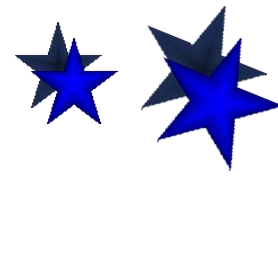
- 非前缀码的例子

$a: 001, b: 00, c: 010, d: 01$

- 解码的歧义，例如字符串 0100001

解码1: 01, 00, 001 d, b, a

解码2: 010, 00, 01 c, b, d



前缀码的二叉树表示

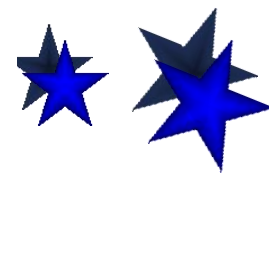
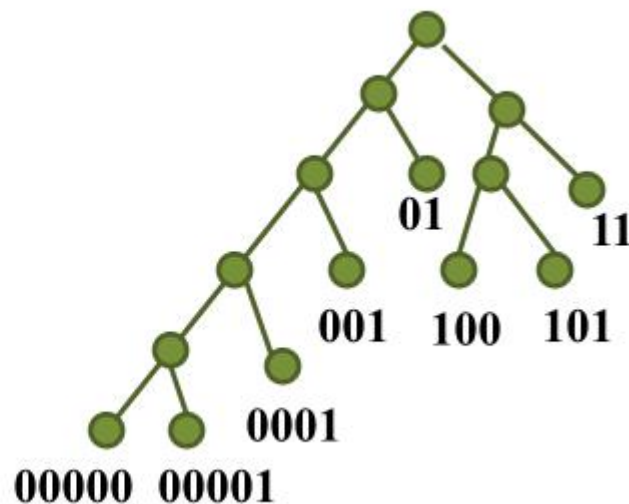
$$\{00000, 00001, 0001, 001, 01, 100, 101, 11\}$$

0-左子树

1-右子树

码对应一片树叶

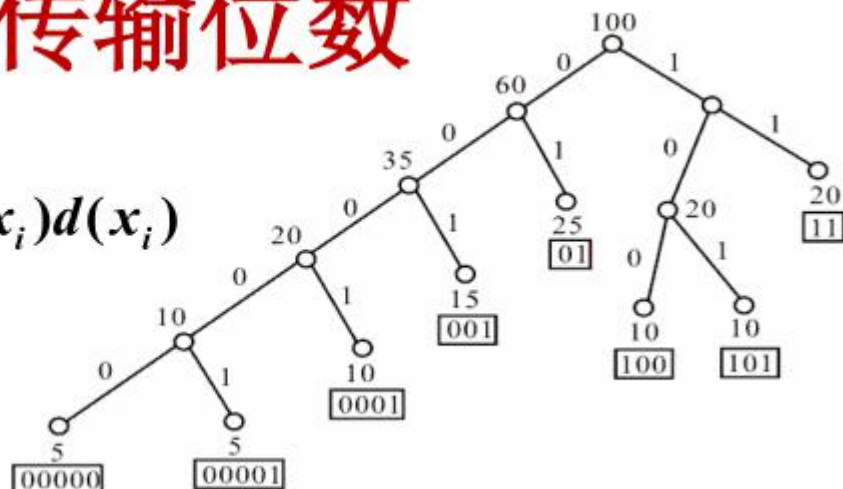
最大位数为树深



4.4 贪心法的典型应用

平均传输位数

$$B = \sum_{i=1}^n f(x_i) d(x_i)$$



$$B = [(5+5) \times 5 + 10 \times 4 + (15+10+10) \times 3 + (25+20) \times 2] / 100 = 2.85$$

问题：给定字符集 $C = \{x_1, x_2, \dots, x_n\}$ 和每个字符的频率 $f(x_i)$, $i=1, 2, \dots, n$. 求关于 C 的一个**最优前缀码**(平均传输位数最小).

4.4 贪心法的典型应用

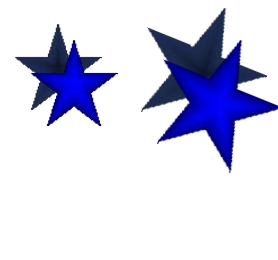
哈夫曼树算法伪码

算法 Huffman(C)

输入: $C = \{x_1, x_2, \dots, x_n\}, f(x_i), i=1, 2, \dots, n.$

输出: Q //队列

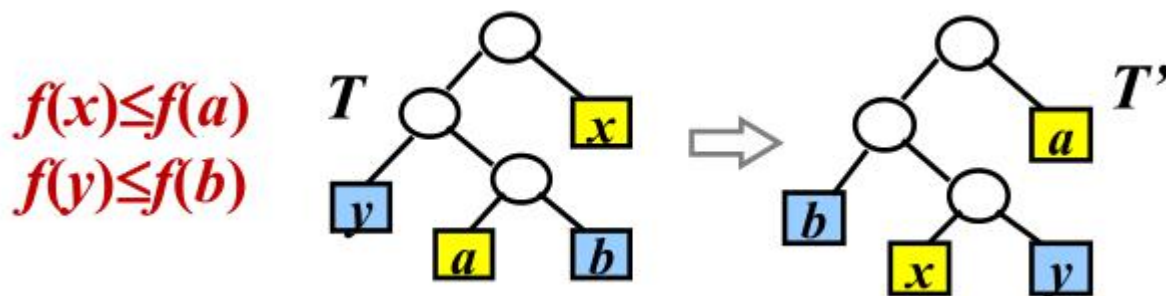
1. $n \leftarrow |C|$
2. $Q \leftarrow C$ //频率递增队列 Q
3. for $i \leftarrow 1$ to $n-1$ do
4. $z \leftarrow \text{Allocate-Node}()$ //生成结点 z
5. $z.\text{left} \leftarrow Q$ 中最小元 //最小作 z 左儿子
6. $z.\text{right} \leftarrow Q$ 中最小元 //最小作 z 右儿子
7. $f(z) \leftarrow f(x) + f(y)$
8. Insert(Q, z) // 将 z 插入 Q
9. return Q



4.4 贪心法的典型应用

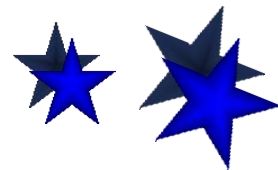
最优前缀码性质：引理1

引理1: C 是字符集, $\forall c \in C, f(c)$ 为频率, $x, y \in C$, $f(x), f(y)$ 频率最小, 那么存在最优二元前缀码使得 x, y 码字等长且仅在最后一位不同.



$$B(T) - B(T') = \sum_{i \in C} f[i]d_T(i) - \sum_{i \in C} f[i]d_{T'}(i) \geq 0$$

其中 $d_T(i)$ 为 i 在 T 中的层数 (i 到根的距离)



4.4 贪心法的典型应用

引理2

引理 设 T 是二元前缀码的二叉树, $\forall x, y \in T$, x, y 是树叶兄弟, z 是 x, y 的父亲, 令

$$T' = T - \{x, y\}$$

且令 z 的频率

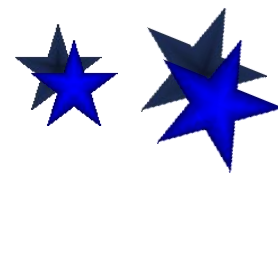
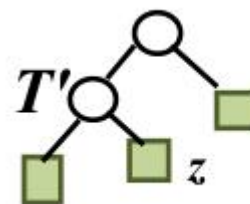
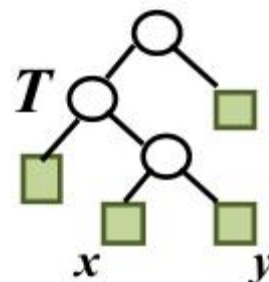
$$f(z) = f(x) + f(y)$$

T' 是对应二元前缀码

$$C' = (C - \{x, y\}) \cup \{z\}$$

的二叉树, 那么

$$B(T) = B(T') + f(x) + f(y)$$



4.4 贪心法的典型应用

引理2证明

证 $\forall c \in C - \{x, y\}$, 有

$$d_T(c) = d_{T'}(c) \Rightarrow f(c)d_T(c) = f(c)d_{T'}(c)$$

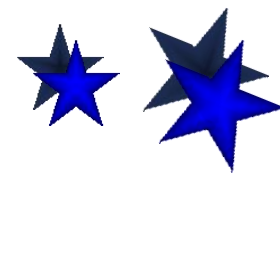
$$d_T(x) = d_T(y) = d_{T'}(z) + 1$$

$$B(T) = \sum_{i \in T} f(i)d_T(i)$$

$$= \sum_{i \in T, i \neq x, y} f(i)d_T(i) + f(x)d_T(x) + f(y)d_T(y)$$

$$= \sum_{i \in T', i \neq z} f(i)d_{T'}(i) + f(z)d_{T'}(z) + (f(x) + f(y))$$

$$= B(T') + f(x) + f(y)$$



4.4 贪心法的典型应用

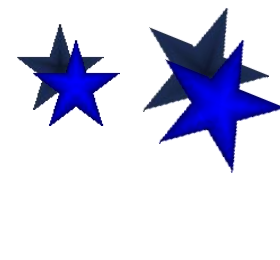
■ 哈夫曼算法的证明及应用

算法正确性证明思路

定理 Huffman 算法对任意规模为 n ($n \geq 2$) 的字符集 C 都得到关于 C 的最优前缀码的二叉树.

归纳基础 证明: 对于 $n=2$ 的字符集, Huffman算法得到最优前缀码.

归纳步骤 证明: 假设Huffman算法对于规模为 k 的字符集都得到最优前缀码, 那么对于规模为 $k+1$ 的字符集也得到最优前缀码.

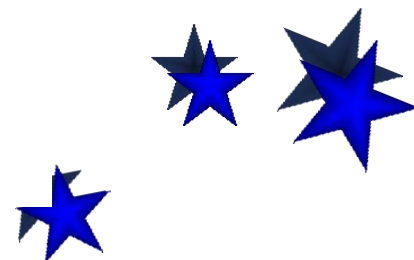
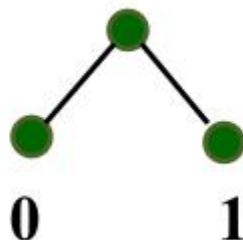


4.4 贪心法的典型应用

归纳基础

$n=2$, 字符集 $C=\{x_1, x_2\}$,

对任何代码的字符至少都需要1位二进制数字. Huffman算法得到的代码是 0 和 1, 是最优前缀码.



4.4 贪心法的典型应用

归纳步骤

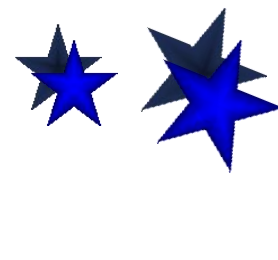
假设Huffman算法对于规模为 k 的字符集都得到最优前缀码. 考虑规模为 $k+1$ 的字符集

$C = \{x_1, x_2, \dots, x_{k+1}\}$,
其中 $x_1, x_2 \in C$ 是频率最小的两个字符.

令

$$C' = (C - \{x_1, x_2\}) \cup \{z\},$$
$$f(z) = f(x_1) + f(x_2)$$

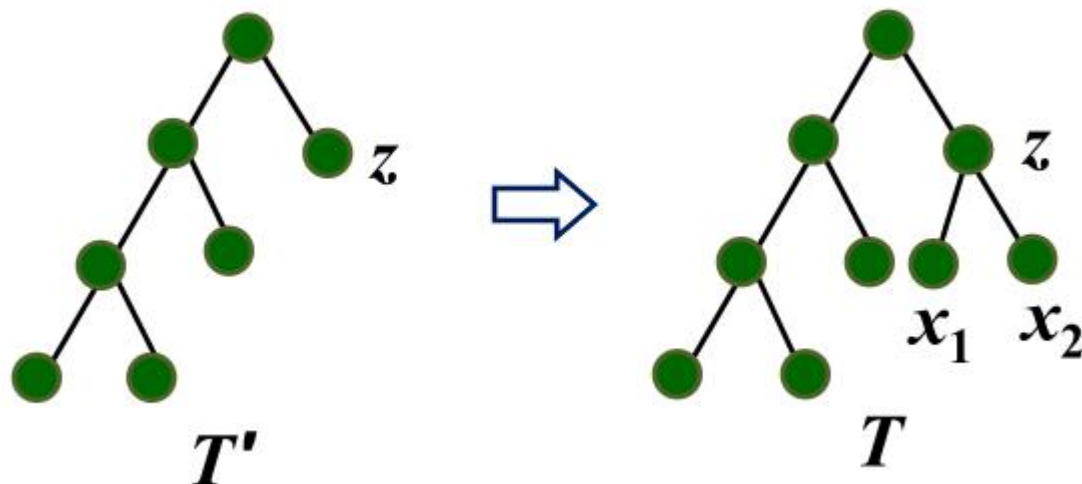
根据归纳假设, 算法得到一棵关于字符集 C' , 频率 $f(z)$ 和 $f(x_i)$ ($i=3, 4, \dots, k+1$) 的最优前缀码的二叉树 T' .



4.4 贪心法的典型应用

归纳步骤(续)

把 x_1, x_2 作为 z 的儿子附到 T' 上, 得到树 T , 那么 T 是关于 $C=(C'-\{z\})\cup\{x_1, x_2\}$ 的最优前缀码的二叉树.



4.4 贪心法的典型应用

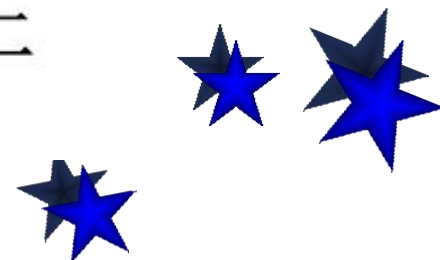
归纳步骤(续)

如若不然, 存在更优树 T^* , $B(T^*) < B(T)$,
且由引理1, 其树叶兄弟是 x_1 和 x_2 .

去掉 T^* 中 x_1 和 x_2 , 得到 $T^{*'}.$ 根据引理2

$$\begin{aligned} B(T^{*'}) &= B(T^*) - \underline{(f(x_1) + f(x_2))} \\ &< B(T) - \underline{(f(x_1) + f(x_2))} \\ &= B(T') \end{aligned}$$

与 T' 是一棵关于 C' 的最优前缀码的二叉树矛盾.



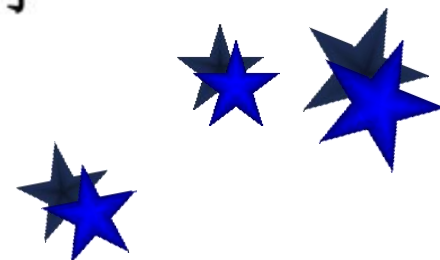
4.4 贪心法的典型应用

应用: 文件归并

问题: 给定一组不同长度的排好序文件构成的集合 $S = \{f_1, \dots, f_n\}$, 其中 f_i 表示第 i 个文件含有的项数. 使用二分归并将这些文件归并成一个有序文件.

归并过程对应于二叉树:

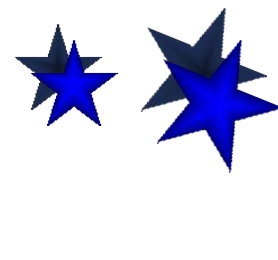
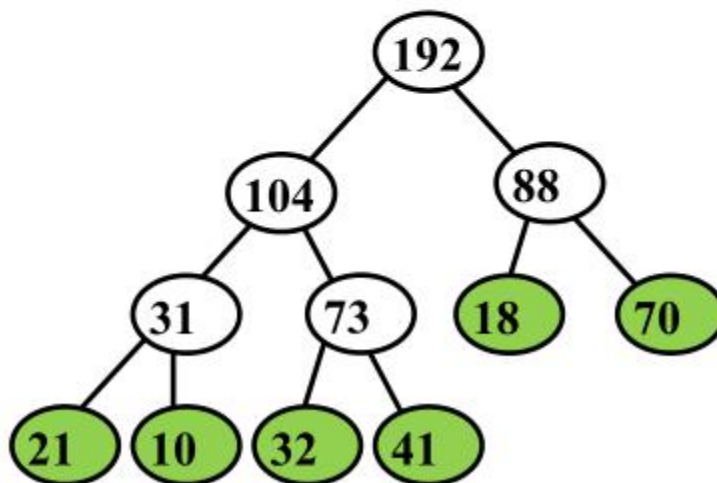
文件为树叶. f_i 与 f_j 归并的文件是它们的父结点.



4.4 贪心法的典型应用

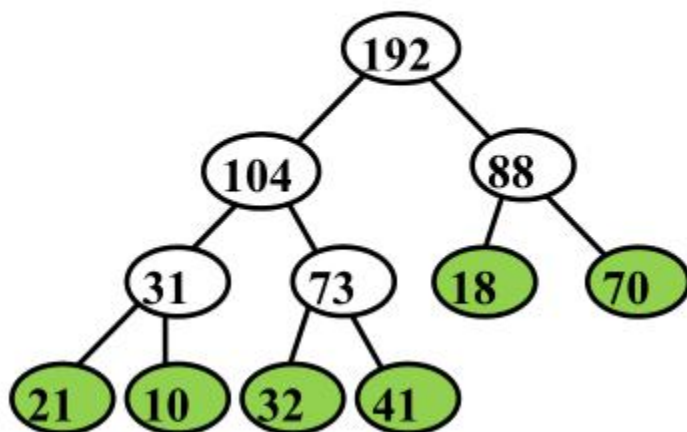
两两顺序归并

实例： $S = \{ 21, 10, 32, 41, 18, 70 \}$



4.4 贪心法的典型应用

归并代价



$$(1) (21+10-1)+(32+41-1)+(18+70-1)+ \\ (31+73-1)+(104+88-1) = 483$$

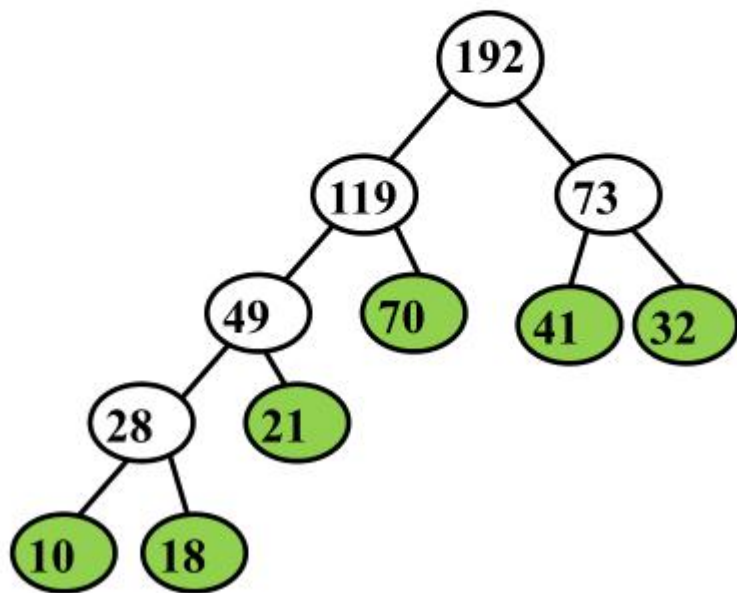
$$(2) (21+10+32+41) \times 3 + (18+70) \times 2 - 5 = 483$$

代价计算公式 $\sum_{i \in S} d(i) f_i - (n-1)$

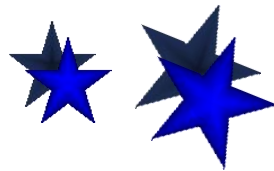
4.4 贪心法的典型应用

实例：Huffman树归并

输入： $S=\{21,10,32,41,18,70\}$



代价： $(10+18) \times 4 + 21 \times 3 + (70+41+32) \times 2 - 5 = 456$



4.4 贪心法的典型应用

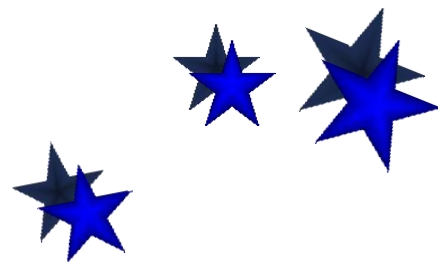
■ 最小生成树

无向连通带权图

$$G = (V, E, W),$$

其中 $w(e) \in W$ 是边 e 的权.

G 的一棵生成树 T 是包含了 G 的所有顶点的树, 树中各边的权之和 $W(T)$ 称为树的权, 具有最小权的生成树称为 G 的最小生成树.

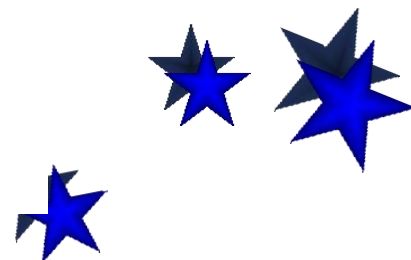
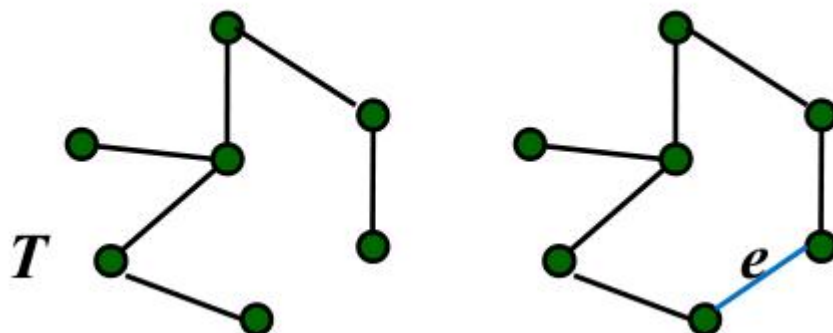


4.4 贪心法的典型应用

生成树的性质

命题1 设 G 是 n 阶连通图, 那么

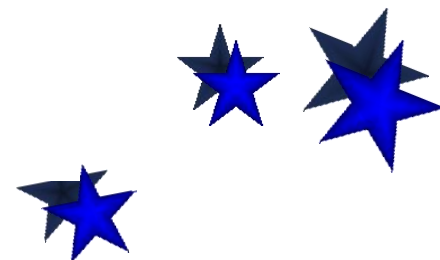
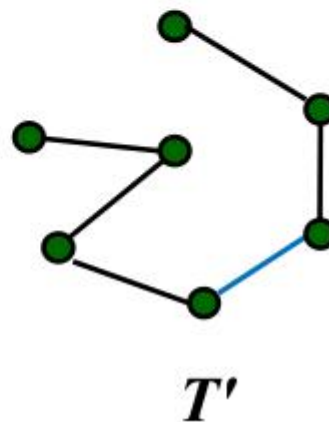
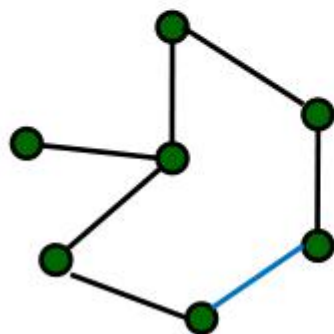
- (1) T 是 G 的生成树当且仅当 T 无圈且有 $n-1$ 条边.
- (2) 如果 T 是 G 的生成树, $e \notin T$, 那么 $T \cup \{e\}$ 含有一个圈 C (回路).



4.4 贪心法的典型应用

生成树的性质（续）

- (3) 去掉圈 C 的任意一条边，就得到 G 的另外一棵生成树 T' 。



4.4 贪心法的典型应用

生成树性质的应用

- 算法步骤：选择边。
约束条件：不形成回路
截止条件：边数达到 $n-1$.
- 改进生成树 T 的方法
在 T 中加一条非树边 e , 形成回路
 C , 在 C 中去掉一条树边 e_i , 形成
一棵新的生成树 T'
$$W(T') - W(T) = W(e) - W(e_i)$$

若 $W(e) \leq W(e_i)$, 则 $W(T') \leq W(T)$

